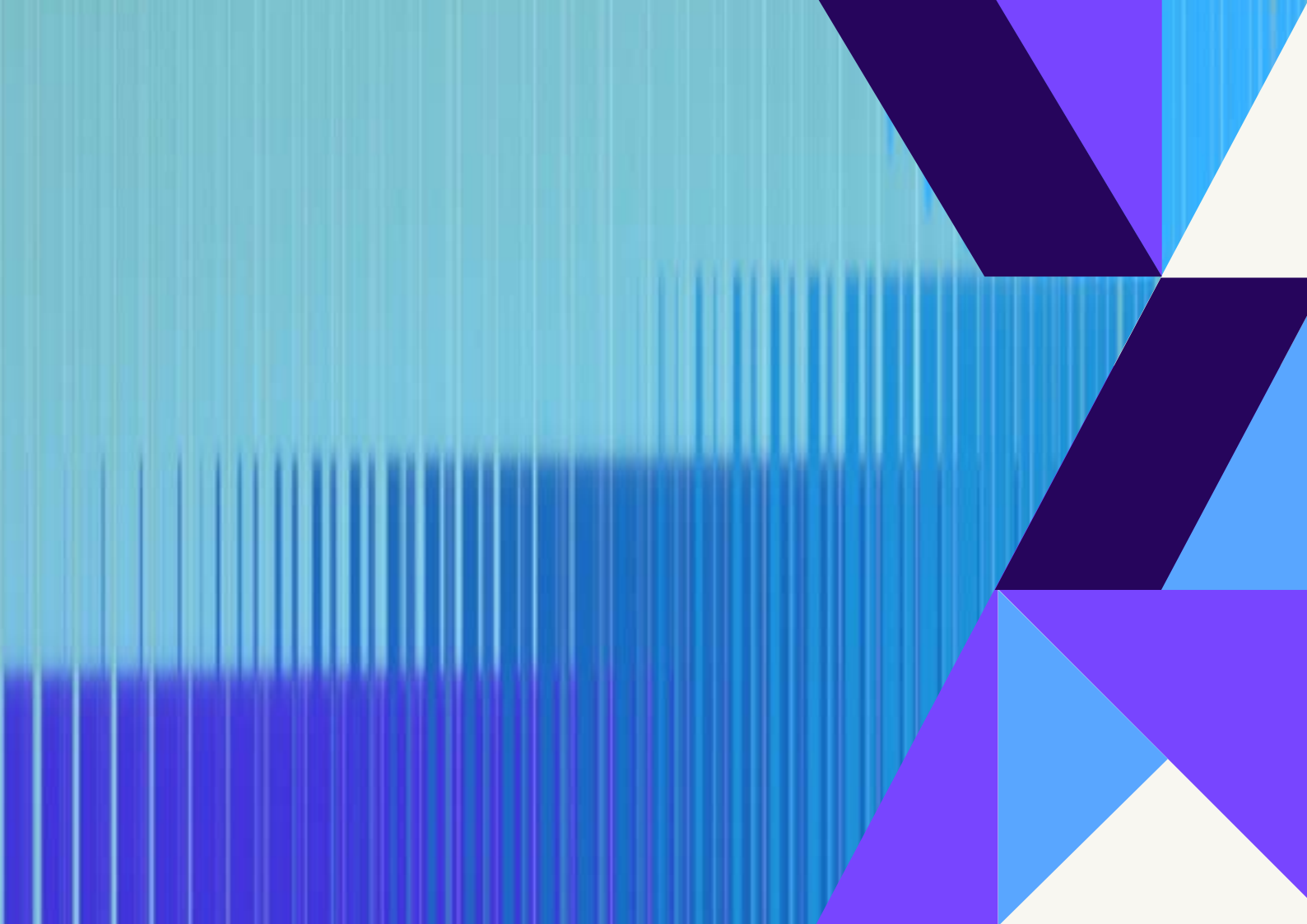




Fundamentos do .NET MAUI

Do primeiro feed ao deploy nas lojas





Sobre o autor

Rafael Saccomani

Olá, sou desenvolvedor e educador com mais de 20 anos de experiência em tecnologia, com forte atuação no desenvolvimento mobile dentro do ecossistema .NET. Sou referência na comunidade brasileira de .NET MAUI, reconhecido como MAUI Show Case pela minha contribuição prática e técnica à plataforma.

Atuo como instrutor no balta, plataforma na qual ajudo a formar a próxima geração de devs .NET focados em mobile, arquitetura e boas práticas.

Compartilho conteúdo técnico de forma constante no canal Papo Saccomani no YouTube, em artigos e em palestras, sempre com uma abordagem direta, prática e baseada em projetos reais.

Acredito que o aprendizado precisa sair do “olá mundo” e chegar ao app publicado, ao código que vai para produção e ao deploy de verdade. Sou pós-graduando em Neurociência, área que me ajuda a entender como as pessoas aprendem e a desenhar materiais e cursos mais eficientes, humanos e duradouros.

Para mim, ensinar é um exercício técnico e cognitivo: programar bem é importante, mas formar profissionais melhores é o que realmente transforma carreiras e times.

Sobre este livro

Há alguns anos, desenvolver um aplicativo para múltiplas plataformas significava escolher entre aprender linguagens distintas, manter codebases separadas ou abrir mão de alguma coisa.

O Xamarin Forms mudou esse cenário para desenvolvedores .NET, e conquistou uma comunidade enorme ao redor do mundo. O .NET MAUI é o próximo passo dessa história. Com ele, você escreve C# e XAML uma única vez e publica o mesmo aplicativo no Android, iOS, Windows e macOS, tudo a partir de um projeto único e bem organizado.

Não é mágica: é a evolução natural de uma plataforma que já provou seu valor na prática. Este livro nasceu de uma premissa simples: aprender fazendo.

Nenhum projeto fictício. Nenhuma arquitetura que existe apenas no slide de uma apresentação. O que você vai construir aqui são interfaces reais, do tipo que você já usou hoje, provavelmente antes mesmo de abrir este arquivo.

O que você vai construir

Ao longo das próximas páginas, três projetos vão guiar sua jornada: Um feed de conteúdo no estilo Instagram ou LinkedIn, onde você vai entender como listas performáticas funcionam no MAUI e como criar uma experiência de scroll fluida e responsiva.

Uma tela de loja virtual com cards modernos, explorando layouts, imagens, tipografia e os detalhes visuais que separam um aplicativo amador de um produto profissional. E, ao final, você vai preparar o app para o mundo real: configurações de publicação, ícones, splash screens e tudo que precisa estar no lugar antes de enviar para as lojas.

Para quem é este livro

Se você já conhece C# e quer entrar no desenvolvimento mobile e desktop sem abandonar o ecossistema .NET, este material foi escrito para você.

Não é necessário experiência anterior com MAUI ou Xamarin. Você vai precisar de curiosidade, de um ambiente configurado e da disposição de escrever código junto com cada capítulo.

Sem atalhos. Sem templates prontos. Apenas prática.

Sumário

Sobre este livro	03
O que você vai construir	04
Para quem é este livro	04
Introdução	06
Construindo um feed de dados	07
Estrutura da Model e ViewModel do feed no .NET MAUI	12
Interface	29
CollectionView.....	29
Header, body e footer	35
Usando DataTemplate	41
O rodapé do card	47
Converters	51
Preparando um aplicativo .NET MAUI para o deploy	61
Conclusão	72

.NET MAUI

O .NET MAUI permite criar aplicativos multiplataforma (Android, iOS, Windows e macOS) usando C# e XAML. É a evolução do Xamarin Forms, framework que conquistou corações e teve milhares de aplicativos lançados.

A maior diferença entre o Maui para Xamarin Forms é a organização única em apenas um projeto e a capacidade de rodar em Desktop Windows e Mac (em breve Linux).

A ideia deste material é simples:

- Criar um feed como Instagram ou LinkedIn
- Preparar o app para publicação
- Criar uma tela estilo loja virtual com cards modernos

Sem complicação. Sem arquitetura pesada. Apenas prática.

Construindo um feed de dados

Todo aplicativo moderno tem um feed. Pode ser uma linha do tempo de fotos, uma lista de notícias, o histórico de pedidos de um e-commerce, eventos de um calendário, mensagens de um chat ou registros internos de um sistema corporativo.

O formato muda, o contexto muda, mas a essência é sempre a mesma: uma sequência de informação que aparece, se organiza e conversa com o usuário em tempo real. O feed virou o coração pulsante das aplicações modernas. É ali que a experiência acontece de verdade.

E é exatamente por isso que ele é um ponto tão importante de aprender da forma correta. Implementar um feed mal construído significa lidar com lentidão, travamentos, consumo excessivo de memória e uma experiência que frustra quem usa.

Implementar bem significa ter uma base sólida que você vai reutilizar em praticamente qualquer projeto. Neste capítulo, vamos construir um feed simples, mas estruturado desde o início da maneira certa.

O que vamos explorar

Não se trata apenas de “mostrar uma lista na tela”.

Qualquer loop faz isso. O que vamos entender aqui é como o .NET MAUI pensa sobre listas, como ele renderiza itens com eficiência e

como conectar dados reais a uma interface visual de forma limpa e sustentável.

Ao longo deste capítulo, você vai trabalhar com:

- **CollectionView**, o componente nativo do MAUI para listas performáticas, projetado para substituir o ListView com muito mais flexibilidade e controle.
- **DataTemplate**, a estrutura que define como cada item da lista será visualmente representado, separando o layout dos dados.
- **Bindings**, o mecanismo que conecta sua interface ao modelo de dados sem que você precise escrever código de atualização manual.
- **Conversores** pequenas peças de lógica que transformam um valor antes de exibi-lo, como converter um booleano em uma cor ou uma data em texto legível.
- **Header e Footer**, os elementos que emolduram o feed acima e abaixo do conteúdo, muito usados para títulos, filtros e indicadores de carregamento.
- **Scroll adequado**, porque um feed que trava ou que se comporta de forma estranha em telas diferentes não serve para produção.

O que estamos construindo

Antes de escrever a primeira linha de código, vale ter clareza sobre o que o feed vai ser. Não em termos de design, mas em termos de estrutura.

Saber o que você está montando antes de montar é o que separa código que cresce bem de código que vira problema em duas semanas.

Nosso feed vai ter cinco características fundamentais.

Um **cabeçalho fixo** no topo, o Header, que pode exibir o título da tela, informações do usuário, filtros ou qualquer dado relevante que precisa estar sempre visível, independente de onde o usuário estiver na lista.

Uma **lista de itens** renderizada com DataTemplate, onde cada elemento segue um layout definido uma única vez e aplicado automaticamente para todos os registros, sejam dez ou dez mil.

Um **rodapé**, o Footer, posicionado ao final do conteúdo. É o lugar natural para indicadores de carregamento, mensagens de “você chegou ao fim” ou botões de paginação.

Um **scroll único e bem estruturado**, onde a página inteira se move como uma unidade coesa. Sem listas dentro de listas. Sem comportamentos conflitantes. Sem o clássico erro de colocar um CollectionView dentro de um ScrollView e passar horas depurando.

E uma **separação clara entre View e dados**, mantendo o XAML responsável pela aparência e o C# responsável pelas informações. Simples na teoria, fundamental na prática.

Visualmente, o resultado vai ser limpo e direto. Mas a estrutura por trás vai ser a mesma que você usaria em um aplicativo de produção. E é justamente essa estrutura que importa aprender.

ViewModel

Se a tela é o palco, a ViewModel é quem organiza tudo nos bastidores.

O usuário nunca vai vê-la. Ela não tem botões, não tem cores, não tem animações. Mas é ela quem decide quais dados aparecem na tela, quando eles são atualizados, como a interface reage a uma

mudança e o que acontece quando alguém toca em algo.

Entender ViewModel no .NET MAUI não é um detalhe técnico reservado para quem quer ir fundo em arquitetura. É fundamento. É o que faz a diferença entre um aplicativo que funciona e um aplicativo que você consegue manter, evoluir e testar sem medo.

Quando você domina esse conceito, sua tela deixa de ser “um arquivo XAML com código atrás” e passa a ser uma peça de uma aplicação estruturada, onde cada responsabilidade tem o seu lugar.

O papel da ViewModel no MVVM

O padrão MVVM divide a aplicação em três camadas com responsabilidades bem definidas.

O **Model** representa os dados. É a estrutura pura da informação: um post, um usuário, um produto. Sem lógica de apresentação, sem conhecimento de interface.

A **View** representa a interface. É o XAML, os layouts, os componentes visuais. Ela sabe como as coisas devem parecer, mas não sabe de onde os dados vêm.

A **ViewModel** faz a ponte entre os dois. Ela pega os dados do Model, os expõe de uma forma que a View consegue consumir, notifica quando algo muda e executa as ações que o usuário dispara.

Na prática, isso significa que a ViewModel expõe propriedades que a View observa, notifica automaticamente quando essas propriedades mudam, executa comandos em resposta a interações e mantém toda a lógica fora do arquivo de interface.

No contexto do nosso feed, ela é o cérebro da operação.

O que vamos construir

Nossa ViewModel vai ser simples e direta, sem cerimônia desnecessária. Três responsabilidades apenas:

Uma **coleção de posts**, que vai alimentar o CollectionView com os itens que aparecem na tela.

Notificação automática de atualização, garantindo que toda vez que a coleção mudar, a interface reflita isso sem que você precise escrever nenhum código de sincronização manual.

Um **comando para carregar os dados**, que pode ser acionado na inicialização da tela ou em resposta a uma ação do usuário, como um botão de recarregar.

Nada além do necessário. Mas tudo no lugar certo.

Estrutura da Model e ViewModel do feed no .NET MAUI

O desenvolvimento de aplicativos modernos exige uma separação clara entre os dados que o sistema manipula e a forma como esses dados são apresentados ao usuário final. No contexto do .NET MAUI, essa separação é implementada pelo padrão arquitetural MVVM (Model-View-ViewModel), que distribui responsabilidades entre três camadas bem definidas e interdependentes.

Antes de escrever qualquer linha de código voltada à interface, é necessário estruturar corretamente as classes que representam os dados e as classes responsáveis por preparar esses dados para exibição. Esta parte aprofunda a construção da Model e da ViewModel utilizadas em um feed de posts, componente central de qualquer aplicativo moderno que precise exibir listas dinâmicas de conteúdo.

O papel da Model no padrão MVVM

A Model é a camada responsável por representar a estrutura dos dados da aplicação.

Ela não contém lógica de apresentação, não conhece a interface gráfica e não possui dependência com nenhum componente visual do .NET MAUI.

No contexto de um feed, a *Model* representa um único item da lista, ou seja, um post completo com todas as propriedades que precisam ser exibidas na tela.

Cada propriedade declarada na *Model* corresponde diretamente a um dado visível no card do feed, como o nome do autor, o título do conteúdo, o texto e a URL da imagem de destaque.

Declaração da classe *Post*

A classe ***Post*** encapsula as informações mínimas necessárias para compor um item do feed.

```
public class Post
{
    public string Author { get; set; }
    public string Title { get; set; }
    public string Content { get; set; }
    public string UrlImage { get; set; }
}
```

A propriedade ***Author*** armazena o nome do autor ou entidade responsável pela publicação do conteúdo.

A propriedade ***Title*** representa o título do post, que será exibido em destaque no card da interface, geralmente com formatação em negrito e tamanho de fonte maior.

A propriedade ***Content*** contém o corpo textual resumido do post, correspondente ao trecho de prévia que o usuário visualiza antes de acessar o conteúdo completo.

A propriedade ***UrlImage*** armazena a URL remota da imagem que encabeça o card do feed, equivalente à foto de destaque presente em plataformas como Instagram ou LinkedIn.

Nesse estágio inicial, todas as propriedades utilizam o tipo **string**, o que mantém a estrutura simples e adequada para cenários onde os dados são carregados a partir de uma API REST ou de qualquer fonte que retorne texto serializado.

Em versões mais avançadas do feed, a Model pode ser expandida com propriedades como **PublishedAt** do tipo **DateTime**, **Tags** do tipo **string** (para uso com converters), e contadores de interações como curtidas e comentários.

A BaseViewModel como contrato de notificação

Antes de analisar a **FeedViewModel**, é necessário compreender a **BaseViewModel**, que é a classe base responsável por implementar a interface **INotifyPropertyChanged**.

Essa interface é o mecanismo pelo qual o .NET MAUI detecta automaticamente quando uma propriedade da ViewModel foi alterada e atualiza a interface sem que o desenvolvedor precise chamar explicitamente nenhum método de refresh.

```
using System.ComponentModel;
using System.Runtime.CompilerServices;

public class BaseViewModel : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler PropertyChanged;

    protected void OnPropertyChanged([CallerMemberName] string
propertyName = null)
    {
        PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(propertyName));
    }
}
```

O atributo `[CallerMemberName]` elimina a necessidade de passar o nome da propriedade como string literal ao chamar `OnPropertyChanged`, pois o compilador injeta automaticamente o nome do membro que originou a chamada.

Essa abordagem reduz o risco de erros causados por refatorações de nomes de propriedades e torna o código mais seguro e legível.

Toda `ViewModel` do projeto deve herdar de `BaseViewModel`, garantindo que qualquer propriedade notificável siga o mesmo contrato de comunicação com a camada de View.

Estrutura da `FeedViewModel`

A `FeedViewModel` é a classe responsável por preparar e expor os dados do feed para a interface gráfica. Ela herda de `BaseViewModel`, possui uma coleção observável de posts, declara um comando de carregamento e inicializa os dados assim que é instanciada.

```
using System.Collections.ObjectModel;
using System.Windows.Input;

public class FeedViewModel : BaseViewModel
{
    public ObservableCollection<Post> Posts { get; set; }

    public ICommand LoadPostsCommand { get; }

    public FeedViewModel()
    {
        Posts = new ObservableCollection<Post>();
        LoadPostsCommand = new Command(LoadPosts);

        LoadPosts();
    }

    private void LoadPosts()
    {
        Posts.Clear();

        Posts.Add(new Post
        {
```

```

        Author = "Equipe",
        Title = "Bem-vindo ao Feed",
        Content = "Este é nosso primeiro post no aplicativo.",
        UrlImage="<https://seusite.com/img001.png>"
    });

    Posts.Add(new Post
    {
        Author = "Sistema",
        Title = "Atualização",
        Content = "CollectionView configurada com sucesso.",
        UrlImage="<https://seusite.com/img002.png>"
    });

    Posts.Add(new Post
    {
        Author = "Admin",
        Title = "Novidade",
        Content = "Em breve teremos paginação e pull to
refresh.",
        UrlImage="<https://seusite.com/img003.png>"
    });
}

```

A estrutura acima, embora minimalista, segue as boas práticas do padrão MVVM e está preparada para evoluir sem necessidade de refatoração estrutural.

Por que utilizar ObservableCollection

A propriedade **Posts** é declarada como **ObservableCollection<Post>**, e essa escolha não é acidental. A **ObservableCollection<T>** é uma coleção especializada do namespace **System.Collections.ObjectModel** que implementa tanto **INotifyCollectionChanged** quanto **INotifyPropertyChanged**.

Isso significa que, sempre que um item é adicionado, removido ou a coleção inteira é substituída, a interface gráfica vinculada a essa coleção é notificada automaticamente e se atualiza sem qualquer intervenção manual.

Uma `List<T>` comum não possui esse comportamento, o que significa que mesmo que o código adicione novos posts à lista, a `CollectionView` na tela não seria capaz de detectar a mudança e o feed permaneceria estático.

A utilização de `ObservableCollection<T>` é, portanto, um requisito funcional em qualquer feed que precise refletir atualizações em tempo real, como o carregamento de novos posts via pull to refresh ou paginação incremental.

O comando `LoadPostsCommand`

A propriedade `LoadPostsCommand` é do tipo `ICommand`, que é a abstração padrão do .NET para representar ações que podem ser acionadas a partir da interface gráfica.

A implementação concreta utilizada é a classe `Command`, disponível diretamente no .NET MAUI, que recebe como parâmetro no construtor o método a ser executado quando o comando for invocado.

Declarar o carregamento de dados como um `ICommand` em vez de chamá-lo diretamente no code-behind da página é uma das premissas centrais do MVVM, pois mantém a lógica de negócio isolada na ViewModel e desacoplada da View.

Isso permite que o mesmo comando seja reutilizado em diferentes contextos da interface, como um botão de recarregamento manual, um gesto de pull to refresh ou até mesmo uma chamada programática a partir de outra ViewModel.

O método LoadPosts e sua responsabilidade

O método privado **LoadPosts** é responsável por popular a coleção `Posts` com os dados que serão exibidos no feed. A chamada a **Posts.Clear()** no início do método garante que nenhum item duplicado seja adicionado em caso de recarregamento, evitando que a lista cresça indefinidamente a cada chamada.

Nesta versão inicial, os dados são estáticos e definidos diretamente no código, o que é adequado para fins de prototipagem e validação do layout.

Em um cenário de produção, esse método seria substituído por uma chamada assíncrona a uma API REST, utilizando **HttpClient** ou um serviço de repositório injetado via construtor, o que permite substituir a fonte de dados sem alterar a estrutura da ViewModel.

O padrão correto para chamadas assíncronas na inicialização de uma ViewModel envolve o uso de **async/await** combinado com um estado de carregamento exposto como propriedade booleana notificável, permitindo que a interface exiba um indicador de progresso enquanto os dados são recuperados.

Inicialização no construtor

No construtor da **FeedViewModel**, três ações ocorrem em sequência: a instância de **ObservableCollection<Post>** é criada e atribuída à propriedade **Posts**, o comando **LoadPostsCommand** é inicializado com a referência ao método **LoadPosts**, e o próprio método **LoadPosts** é chamado imediatamente.

A chamada direta a `LoadPosts()` no construtor garante que o feed já exiba conteúdo no momento em que a tela é renderizada pela primeira vez.

Essa abordagem é válida para dados síncronos ou para cenários de dados estáticos, mas deve ser substituída por um padrão de inicialização assíncrona quando os dados precisarem ser recuperados de uma fonte externa.

Vinculando a ViewModel à Page

Para que o binding funcione corretamente, a ViewModel precisa ser associada ao `BindingContext` da página correspondente.

Essa associação é feita no construtor da `ContentPage`, conforme demonstrado a seguir:

```
namespace Feed;

public partial class MainPage : ContentPage
{
    public MainPage()
    {
        InitializeComponent();
        BindingContext = new FeedViewModel();
    }
}
```

O `BindingContext` é a propriedade fundamental do sistema de binding do .NET MAUI que define a origem dos dados para todos os elementos visuais contidos na página.

Ao definir `BindingContext = new FeedViewModel()`, todos os controles XAML da página que utilizarem expressões `{Binding NomeDaPropriedade}` passarão a buscar seus valores diretamente na instância da `FeedViewModel`.

Essa é a forma mais simples de realizar a vinculação, adequada para projetos de pequeno e médio porte que não utilizam um container de injeção de dependência.

Em projetos maiores, especialmente aqueles baseados em Clean Architecture, é recomendado injetar a ViewModel via construtor utilizando um container de DI como o **Microsoft.Extensions.DependencyInjection**, disponível nativamente no .NET MAUI a partir do **Mauiprogram.cs**.

Escalabilidade da estrutura

A estrutura apresentada, composta pela classe **Post**, pela **BaseViewModel** e pela **FeedViewModel**, representa a base mínima necessária para um feed funcional e corretamente organizado.

Esse núcleo pode ser expandido progressivamente à medida que os requisitos do projeto evoluem, sem que seja necessário reescrever as classes existentes.

A adição de novas propriedades à Model, como **Tags**, **PublishedAt** ou **LikesCount**, não quebra nenhum contrato existente e pode ser feita de forma incremental.

Da mesma forma, a **FeedViewModel** pode ser estendida com novas propriedades de estado, como **IsLoading**, **HasError** e **IsEmpty**, cada uma notificando a interface via **OnPropertyChanged`** para garantir uma experiência de usuário responsiva e consistente.

Um pouco mais sobre BaseViewModel

O desenvolvimento de aplicativos .NET MAUI estruturados sobre o padrão MVVM exige que todas as ViewModels compartilhem um conjunto comum de comportamentos relacionados à notificação de mudanças de estado.

Sem uma classe base centralizada, cada ViewModel precisaria implementar individualmente a interface **INotifyPropertyChanged**, o que gera duplicação de código, aumenta o risco de inconsistências e dificulta a manutenção do projeto à medida que ele cresce.

A **BaseViewModel** resolve esse problema de forma elegante, fornecendo um contrato unificado de notificação que todas as ViewModels do projeto herdam automaticamente.

O problema que a BaseViewModel resolve

Imagine um projeto com dezenas de telas, cada uma com sua própria ViewModel.

Sem uma classe base, cada uma dessas ViewModels precisaria declarar o evento **PropertyChanged**, implementar o método de disparo e gerenciar o nome da propriedade alterada, repetindo o mesmo bloco de código em todos os arquivos.

Além da redundância, qualquer alteração nesse comportamento, como a adição de logs de diagnóstico ou um mecanismo de rastreamento de estado, exigiria a modificação de cada

ViewModel individualmente.

A `BaseViewModel` centraliza essa responsabilidade em um único lugar, tornando qualquer mudança futura no comportamento de notificação uma alteração pontual e de baixo risco.

A interface `INotifyPropertyChanged`

A `INotifyPropertyChanged` é uma interface nativa do .NET, declarada no namespace `System.ComponentModel`, que define um único membro: o evento `PropertyChanged`.

Quando uma propriedade de uma ViewModel é alterada e o evento `PropertyChanged` é disparado com o nome correto da propriedade, o sistema de binding do .NET MAUI detecta essa notificação e atualiza automaticamente todos os controles visuais vinculados àquela propriedade.

Sem essa interface implementada corretamente, as propriedades da ViewModel não comunicam suas mudanças à interface gráfica, o que significa que atualizações em tempo real no feed, como a chegada de novos posts ou a mudança de estado de um botão, simplesmente não seriam refletidas na tela.

Essa interface é, portanto, o mecanismo fundamental sobre o qual todo o sistema reativo do .NET MAUI se apoia.

Declaração da `BaseViewModel`

A implementação da `BaseViewModel` segue uma estrutura mínima e suficiente para cobrir os requisitos de notificação da

grande maioria dos cenários.

```
using System.ComponentModel;
using System.Runtime.CompilerServices;

public class BaseViewModel : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler PropertyChanged;

    protected void OnPropertyChanged([CallerMemberName] string
propertyName = null)
    {
        PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(propertyName));
    }
}
```

Essa classe declara o evento **PropertyChanged** exigido pela interface e fornece o método protegido **OnPropertyChanged**, que encapsula a lógica de disparo desse evento.

O modificador de acesso **protected** é intencional: ele garante que apenas as classes derivadas possam chamar esse método, evitando que código externo dispare notificações de propriedade indevidas sobre a ViewModel.

O atributo CallerMemberName

O ponto mais relevante da implementação acima é o uso do atributo **[CallerMemberName]** no parâmetro **propertyName** do método **OnPropertyChanged**.

Esse atributo, disponível no namespace **System.Runtime.CompilerServices**, instrui o compilador C# a injetar automaticamente, em tempo de compilação, o nome do membro que originou a chamada ao método.

Na prática, isso significa que ao chamar **OnPropertyChanged()** dentro de uma propriedade da ViewModel, o compilador substitui

o valor padrão `null` pelo nome exato daquela propriedade, sem que o desenvolvedor precise passá-lo como string. O benefício direto dessa abordagem é a eliminação de strings literais no código, que são invisíveis ao compilador e ao sistema de refatoração do Visual Studio.

Em abordagens mais antigas, sem `[CallerMemberName]`, era comum ver código como `OnPropertyChanged("Title")`, onde um simples erro de digitação ou uma renomeação de propriedade sem atualização do argumento gerava um bug silencioso: a interface parava de atualizar sem nenhuma mensagem de erro. Com `[CallerMemberName]`, esse problema deixa de existir, pois o nome da propriedade é sempre resolvido pelo compilador e permanece sincronizado com qualquer refatoração realizada via IDE.

Como uma ViewModel herda e utiliza a BaseViewModel

A `FeedViewModel`, apresentada no capítulo anterior, herda de `BaseViewModel` e automaticamente ganha o contrato de notificação implementado. Para que uma propriedade notifique a interface quando seu valor muda, ela precisa ser declarada com um campo de apoio privado e chamar `OnPropertyChanged()` no setter. O exemplo a seguir demonstra uma propriedade `IsLoading` que poderia ser adicionada à `FeedViewModel` para controlar a exibição de um indicador de carregamento:

```
private bool _isLoading;
public bool IsLoading
{
    get => _isLoading;
    set
    {
        _isLoading = value;
        OnPropertyChanged();
    }
}
```


Nesse padrão, o getter retorna o valor do campo privado e o setter atualiza o campo e dispara a notificação.

A chamada `OnPropertyChanged()` sem argumentos funciona corretamente porque o atributo `[CallerMemberName]` resolve o nome `"IsLoading"` automaticamente em tempo de compilação.

Quando o setter for executado e o evento `PropertyChanged` for disparado, qualquer controle XAML vinculado a `IsLoading` atualizará seu estado imediatamente, como a visibilidade de um `ActivityIndicator` ou a habilitação de um botão.

O operador de propagação nula no disparo do evento

No corpo do método `OnPropertyChanged`, o evento é disparado com a expressão `PropertyChanged?.Invoke(...)`. O operador `?.` é o operador de propagação nula do C#, que verifica se o evento possui ao menos um assinante antes de tentar invocá-lo. Sem esse operador, tentar invocar o evento quando nenhum assinante estiver registrado resultaria em uma `NullReferenceException` em tempo de execução.

Esse detalhe é especialmente relevante em testes unitários de ViewModels, onde o `BindingContext` pode não estar configurado e, portanto, nenhum assinante estará registrado no evento `PropertyChanged`.

Extensões comuns da BaseViewModel

A `BaseViewModel`` pode ser estendida ao longo do projeto para incluir comportamentos compartilhados por todas as

ViewModels sem afetar as implementações existentes. Um padrão muito comum é adicionar uma propriedade **IsBusy** que sinaliza que a ViewModel está executando uma operação assíncrona, e uma propriedade **Title** que representa o título da tela correspondente.

Outra extensão frequente é um método auxiliar que combina a atualização do campo de apoio com o disparo da notificação e o retorno de um booleano indicando se o valor efetivamente mudou, evitando notificações desnecessárias quando o setter é chamado com o mesmo valor que já estava armazenado.

Esse padrão pode ser implementado da seguinte forma:

```
protected bool SetProperty<T>(ref T backingField, T value,
[CallerMemberName] string propertyName = null)
{
    if (EqualityComparer<T>.Default.Equals(backingField, value))
        return false;

    backingField = value;
    OnPropertyChanged(propertyName);
    return true;
}
```

Com esse método auxiliar, os setters das propriedades ficam ainda mais concisos:

```
private bool _isLoading;
public bool IsLoading
{
    get => _isLoading;
    set => SetProperty(ref _isLoading, value);
}
```

Essa é exatamente a abordagem adotada pelo **Community-Toolkit.Mvvm**, uma biblioteca mantida pela Microsoft que oferece uma **BaseViewModel** completa chamada **ObservableObject**, além de source generators que eliminam comple-

tamente o boilerplate das propriedades notificáveis via atributo **ObservableProperty**.

BaseViewModel e testabilidade

Uma das vantagens frequentemente subestimadas de manter a lógica nas ViewModels, em vez de no code-behind das páginas, é a testabilidade.

Como a **BaseViewModel** e todas as suas classes derivadas são classes C# puras sem dependência direta de componentes visuais, elas podem ser instanciadas e testadas em projetos de teste unitário sem a necessidade de um emulador ou dispositivo físico.

Um teste unitário pode verificar, por exemplo, se a propriedade **Posts** da **FeedViewModel** contém o número correto de itens após a chamada ao método de carregamento, ou se a propriedade **IsLoading** transita corretamente entre **true** e **false** durante uma operação assíncrona.

Essa capacidade de testar a lógica de apresentação de forma isolada é um dos pilares que torna o padrão MVVM superior ao padrão code-behind em projetos de médio e grande porte.

Relação com o restante do feed

No contexto da construção do feed apresentada neste material, a **BaseViewModel** é o ponto de partida invisível que sustenta toda a reatividade da interface. A **FeedViewModel** herda dela e, por isso, qualquer propriedade notificável que for adicionada ao feed, como um estado de carregamento, uma mensagem de erro ou um indicador de lista vazia, comunicará suas mudanças à **CollectionView** e aos demais controles da tela de forma automática e confiável.

A **ObservableCollection<Post>** cobre as notificações de mudança na coleção em si, mas qualquer propriedade escalar da ViewModel, como um **bool**, um **string** ou um **int**, depende inteiramente do mecanismo fornecido pela **BaseViewModel** para ser reativa.

Interface

CollectionView

A exibição de listas é uma das operações mais comuns em aplicativos móveis modernos.

Feeds de conteúdo, históricos de pedidos, catálogos de produtos e timelines de notificações são todos, em essência, listas de itens renderizados sequencialmente na tela.

No .NET MAUI, o componente projetado para lidar com esse tipo de exibição de forma eficiente, flexível e completamente customizável é a **CollectionView**.

A evolução a partir do ListView

Durante a era do Xamarin.Forms, o componente padrão para exibição de listas era o **ListView**.

Ele cumpriu bem seu papel por anos, mas carregava limitações estruturais difíceis de contornar: performance degradada em listas longas, dificuldades com layouts personalizados, dependência de células específicas como **TextCell** e **ImageCell**, e ausência de suporte nativo a layouts em grade.

O .NET MAUI marcou a descontinuação formal do **ListView** e posicionou a **CollectionView** como seu substituto definitivo.

A **CollectionView** foi projetada desde o início com foco em performance, flexibilidade e extensibilidade, resolvendo estruturalmente as limitações que o **ListView** acumulou ao longo do tempo.

Migrar qualquer projeto que ainda utilize **ListView** para **CollectionView** é uma decisão técnica recomendada, e novos projetos não devem utilizar **ListView** em nenhuma circunstância.

Virtualização e gerenciamento de memória

A característica técnica mais importante da **CollectionView** é sua preparação nativa para virtualização de itens.

Virtualização significa que o componente não renderiza simultaneamente todos os elementos da coleção vinculada, independentemente de quantos itens ela contenha.

Em vez disso, apenas os itens visíveis na área da tela são efetivamente instanciados e renderizados, enquanto os itens fora da área visível são descartados ou mantidos em um pool de reuso para serem aproveitados conforme o usuário realiza o scroll.

Esse mecanismo tem implicações diretas na performance e no consumo de memória:

um feed com quinhentos posts consome praticamente a mesma quantidade de memória que um feed com dez posts, pois a **CollectionView** gerencia o ciclo de vida dos elementos visuais de forma inteligente.

Sem virtualização, como ocorre em layouts que empilham todos os itens diretamente dentro de um **ScrollView** com **VerticalStackLayout**, cada item é instanciado e mantido em memória simultaneamente, o que leva rapidamente ao consumo excessivo de recursos e a uma experiência de scroll travada e inconsistente.

Compatibilidade com múltiplos layouts

A **CollectionView** suporta diferentes configurações de layout

por meio da propriedade **ItemsLayout**. O layout padrão, quando nenhum valor é especificado, é uma lista vertical com um item por linha, equivalente ao comportamento clássico de um feed.

Para um layout horizontal, onde os itens são exibidos lado a lado e o scroll ocorre da esquerda para a direita, utiliza-se **“ItemsLayout=“HorizontalList”**. Para um layout em grade, onde os itens são organizados em colunas, utiliza-se **“LinearItemsLayout”** ou **“GridItemsLayout”** com a quantidade de colunas desejada.

Essa flexibilidade permite que um único componente atenda a cenários completamente distintos: um feed vertical de posts, uma galeria horizontal de imagens e um catálogo em grade de produtos podem ser todos implementados com **CollectionView**, variando apenas a configuração de layout.

Estrutura básica da CollectionView

A declaração mínima de uma **CollectionView** vinculada a uma coleção de dados exige apenas a definição da propriedade **ItemsSource** e a estrutura do **ItemTemplate**.

```
<CollectionView ItemsSource="{Binding Posts}">
  <CollectionView.ItemTemplate>
    <DataTemplate>
      <Frame Padding="10" Margin="5">
        <VerticalStackLayout>
          <Label Text="{Binding Title}"
FontAttributes="Bold"/>
          <Label Text="{Binding Description}" />
        </VerticalStackLayout>
      </Frame>
    </DataTemplate>
  </CollectionView.ItemTemplate>
</CollectionView>
```

A propriedade **ItemsSource** recebe um binding apontando para a propriedade **Posts** da **FeedViewModel**, que é uma **ObservableCollection<Post>**.

Cada vez que um item é adicionado ou removido dessa coleção, a **CollectionView** detecta a mudança via **INotifyCollectionChanged** e atualiza a interface automaticamente.

A propriedade **ItemTemplate** define a estrutura visual que será replicada para cada item da coleção, e seu conteúdo obrigatório é sempre um **DataTemplate**.

O papel do DataTemplate

O **DataTemplate** é a peça central que separa a estrutura visual de um item dos dados que ele representa.

Dentro do **DataTemplate**, o **BindingContext** é automaticamente configurado pelo .NET MAUI para apontar para o objeto correspondente àquele item da coleção, que no caso do feed é uma instância da classe **Post**.

Isso significa que expressões como **{Binding Title}** e **{Binding Description}** dentro do **DataTemplate** resolvem automaticamente para as propriedades **Title** e **Description** do objeto **Post** correspondente, sem necessidade de qualquer configuração adicional.

Esse mecanismo é o que permite que um único **DataTemplate** seja reutilizado para renderizar dezenas, centenas ou milhares de itens diferentes, cada um com seus próprios dados, mantendo a mesma estrutura visual.

O controle Frame como container de card

No exemplo básico, o **Border** é utilizado como container do card de cada item do feed.

O **Border** é um controle de layout do .NET MAUI que adiciona bordas, cantos arredondados e sombra ao seu conteúdo, sendo adequado para criar a aparência de um card elevado.

A propriedade **Padding** define o espaçamento interno entre a borda do **Border** e seu conteúdo, enquanto **Margin** define o espaçamento externo entre o **Border** e os elementos adjacentes na lista.

As propriedades **StrokeShape** e **StrokeThickness** definem o comportamento do arredondamento e a grossura da linha do nosso border.

Vinculação com a ViewModel

Para que a **CollectionView** funcione corretamente, a página que a contém precisa ter seu **BindingContext** configurado para a **FeedViewModel**.

Essa configuração, feita no construtor da **ContentPage**, é o que permite que o binding **{Binding Posts}** na propriedade **ItemsSource** resolva corretamente para a coleção de posts exposta pela ViewModel.

Sem essa configuração, o binding não encontraria nenhuma propriedade **Posts** para se vincular e a **CollectionView** seria renderizada vazia, sem nenhuma mensagem de erro visível, o que é um dos erros mais comuns para desenvolvedores iniciando com o padrão MVVM.

Recursos adicionais da CollectionView

Além da exibição de listas simples, a **CollectionView** oferece uma série de recursos nativos que cobrem os casos de uso mais comuns em aplicativos modernos.

O suporte nativo a **EmptyView** permite definir um conteúdo alternativo a ser exibido quando a coleção vinculada está vazia, eliminando a necessidade de controlar manualmente a visibilidade de um placeholder.

O suporte a **Header** e **Footer** permite adicionar conteúdo fixo antes e após a lista sem recorrer ao uso de múltiplos **ScrollView**, que geraria conflitos de rolagem e problemas de performance.

O suporte a seleção de itens, controlado pela propriedade **SelectionMode**, permite implementar seleção única ou múltipla com binding direto para a ViewModel, sem precisar de event handlers no code-behind.

O suporte a **RefreshView** como container permite implementar o gesto de pull to refresh de forma nativa, conectado a um **ICommand** da ViewModel que executa o recarregamento dos dados.

CollectionView versus ScrollView com VerticalStackLayout

Um padrão incorreto frequentemente utilizado por desenvolvedores que estão iniciando no .NET MAUI é empilhar controles dentro de um **ScrollView** com um **VerticalStackLayout** em vez de utilizar a **CollectionView**.

Essa abordagem pode parecer mais simples à primeira vista, mas apresenta problemas sérios de performance em listas com mais de algumas dezenas de itens, pois todos os elementos são instanciados e mantidos em memória simultaneamente, sem qualquer virtualização.

Além disso, o uso de **CollectionView** dentro de um **ScrollView** cria conflitos de rolagem que resultam em comportamentos imprevisíveis e difíceis de depurar.

A regra prática é clara: sempre que o objetivo for renderizar uma coleção de itens de quantidade variável ou potencialmente grande, a **CollectionView** é o componente correto e o **ScrollView** com empilhamento manual de itens é o antipadrão a ser evitado.

Preparação para recursos avançados do feed

A estrutura básica da **CollectionView** apresentada neste capítulo é o ponto de partida para a construção de todos os recursos avançados do feed.

O **DataTemplate** será evoluído para incluir imagens, tags com cores dinâmicas via converters e informações de autor, conforme detalhado nos capítulos subsequentes deste material.

O **Header** e o **Footer** serão utilizados para exibir um título fixo no topo do feed e um indicador de carregamento no rodapé, sem a necessidade de qualquer **ScrollView** externo.

Os **IValueConverter** serão integrados diretamente nas expressões de binding dentro do **DataTemplate** para transformar dados brutos da Model em representações visuais adequadas, como a conversão de uma string de tags separadas por vírgula em coleções de pills coloridos.

Header, body e footer da CollectionView

Uma das decisões de layout mais impactantes no desenvolvimento de aplicativos .NET MAUI está relacionada a como o conteúdo fixo é posicionado antes e após uma lista de itens.

A solução incorreta para esse problema é tão comum que merece atenção especial: empilhar uma **CollectionView** dentro de um **ScrollView** junto com outros controles, criando múltiplos contextos de rolagem em uma mesma tela. Uma das decisões de layout mais impactantes no desenvolvimento de aplicativos .NET

MAUI está relacionada a como o conteúdo fixo é posicionado antes e após uma lista de itens.

A solução incorreta para esse problema é tão comum que merece atenção especial: empilhar uma **CollectionView** dentro de um **ScrollView** junto com outros controles, criando múltiplos contextos de rolagem em uma mesma tela.

O resultado desse antipadrão é um feed com comportamento de scroll imprevisível, conflitos de gestos entre os componentes de rolagem e degradação severa de performance, especialmente em dispositivos Android.

A solução correta está disponível nativamente na própria **CollectionView**, por meio das propriedades **Header** e **Footer**

O problema dos múltiplos ScrollViews

Quando um desenvolvedor precisa exibir um título acima de uma lista e um botão de carregamento abaixo dela, a primeira tentativa costuma ser encapsular tudo dentro de um **ScrollView**.

Essa abordagem resulta em uma estrutura onde o **ScrollView** externo tenta controlar a rolagem da tela inteira, enquanto a **CollectionView** interna possui seu próprio mecanismo de scroll e virtualização.

Esse conflito se manifesta de formas distintas dependendo da plataforma: no Android, a **CollectionView** frequentemente colapsa para uma altura mínima dentro do **ScrollView**, exibindo apenas um ou dois itens e forçando o usuário a rolar dentro de um espaço minúsculo.

No iOS, o comportamento pode ser diferente, mas igualmente problemático, com gestos de scroll sendo capturados alternadamente por um componente ou pelo outro de forma inconsistente.

Além dos problemas visuais, a virtualização da **CollectionView** é completamente anulada quando ela é colocada dentro de um **ScrollView**, pois o componente externo precisa conhecer a altura total do conteúdo para calcular seu próprio scroll, o que força a instanciação de todos os itens da coleção simultaneamente.

A solução nativa: Header e Footer

A **CollectionView** foi projetada com suporte nativo a conteúdo fixo posicionado antes e após a lista de itens, por meio das propriedades **Header** e **Footer**.

```
<CollectionView ItemsSource="{Binding Posts}">
    <CollectionView.Header>
        <Label Text="Últimas Atualizações"
              FontSize="20"
              Margin="10"/>
    </CollectionView.Header>
    <CollectionView.Footer>
        <Label Text="Carregando mais..."
              HorizontalOptions="Center"
              Margin="10"/>
    </CollectionView.Footer>
</CollectionView>
```

Com essa abordagem, o **Header**, os itens da lista e o **Footer** fazem parte de um único contexto de rolagem gerenciado exclusivamente pela **CollectionView**.

O **Header** é renderizado acima do primeiro item da lista e rola junto com ela quando o usuário realiza o scroll para baixo, criando o efeito de que o cabeçalho desaparece naturalmente conforme o feed é consumido.

O **Footer** é renderizado abaixo do último item e se torna visível quando o usuário chega ao final da lista, sendo o local ideal para indicadores de carregamento de novos itens ou mensagens de fim de conteúdo.

Em nenhum momento é necessário um **ScrollView** externo, pois toda a rolagem é controlada por um único componente.

Anatomia do Header

O **Header** aceita qualquer estrutura de layout XAML como conteúdo, não se limitando a um simples **Label**. É possível declarar um **StackLayout** completo com imagens, títulos, subtítulos e botões, transformando o cabeçalho do feed em um espaço funcional e informativo.

Um caso de uso comum é posicionar botões de filtro e ordenação no **Header**, permitindo que o usuário reordene o feed por data, relevância ou categoria sem precisar navegar para outra tela.

Outro uso frequente é exibir um resumo estatístico no topo da lista, como o número total de resultados encontrados em uma busca ou o valor total de uma lista de pedidos.

Em feeds com imagens de destaque ou banners promocionais, o **Header** é o local natural para esse conteúdo, garantindo que ele apareça acima dos itens do feed sem duplicar o componente de scroll.

A propriedade **HeaderTemplate** aceita um **DataTemplate** completo, o que permite vincular o conteúdo do cabeçalho a uma propriedade da ViewModel via binding, tornando o header dinâmico e reativo a mudanças de estado.

Anatomia do Footer

O **Footer** é posicionado abaixo do último item da lista e permanece invisível até que o usuário role até o final do feed.

Esse posicionamento o torna ideal para indicadores de carregamento assíncrono de novos itens, que devem aparecer apenas quando o conteúdo atual foi totalmente consumido.

Um padrão amplamente utilizado em aplicativos de redes sociais e e-commerce é exibir um **ActivityIndicator** no **Footer** enquanto o próximo lote de dados está sendo carregado da API, e substituí-lo por uma mensagem de fim de conteúdo quando não há mais itens disponíveis. Esse comportamento pode ser implementado com binding direto na visibilidade do indicador de carregamento, conectado a uma propriedade booleana **IsLoadingMore** da **ViewModel** que é notificável via **BaseViewModel**.

Em listas com paginação, o **Footer** também é o local adequado para um botão de carregamento manual, caso o carregamento automático via **RemainingItemsThreshold** não seja o comportamento desejado. A propriedade **FooterTemplate** aceita um **DataTemplate** da mesma forma que **HeaderTemplate**, permitindo vinculação direta com a **ViewModel**.

O body como a área de itens

O body da **CollectionView** não possui uma propriedade explícita com esse nome: ele é simplesmente a área onde os itens da coleção são renderizados, definidos pelo **ItemTemplate**.

A separação conceitual entre header, body e footer existe para comunicar que a **CollectionView** é, na prática, um container de scroll completo e autossuficiente que possui três zonas distintas de conteúdo. Compreender essa divisão é fundamental para evitar o antipadrão do **ScrollView** externo, pois deixa claro que todo o conteúdo relacionado ao feed deve residir dentro da **CollectionView**, e não ao redor dela.

Header fixo versus header rolável

Por padrão, o **Header** da **CollectionView** rola junto com os itens, desaparecendo da vista conforme o usuário avança no feed. Esse comportamento é adequado para a maioria dos cenários, especialmente quando o cabeçalho contém conteúdo informativo ou

decorativo que não precisa permanecer visível o tempo todo. Em casos onde o cabeçalho precisa ser fixo na tela independentemente da posição de scroll, como uma barra de busca ou um conjunto de filtros que deve estar sempre acessível, a solução correta no .NET MAUI não é utilizar o **Header** da **CollectionView**. Para conteúdo verdadeiramente fixo acima da lista, o padrão recomendado é posicionar o elemento fixo em um **Grid** ou **VerticalStackLayout** fora da **CollectionView**, garantindo que ele não faça parte do contexto de scroll.

A **CollectionView** ocuparia então apenas a área restante da tela abaixo do elemento fixo, utilizando **VerticalOptions="FillAndExpand"** ou o sistema de linhas do **Grid** para preencher o espaço disponível.

Uso combinado com DataTemplate

O **Header** e o **Footer** coexistem naturalmente com o **ItemTemplate** que define o layout de cada item do feed. A estrutura completa de uma **CollectionView** com as três zonas preenchidas pode ser visualizada da seguinte forma:

```
<CollectionView ItemsSource="{Binding Posts}">
    <CollectionView.Header>
        <Label Text="Últimas Atualizações"
              FontSize="20"
              Margin="10"/>
    </CollectionView.Header>

    <CollectionView.ItemTemplate>
        <DataTemplate>
            <!-- estrutura visual de cada item do feed -->
        </DataTemplate>
    </CollectionView.ItemTemplate>

    <CollectionView.Footer>
        <Label Text="Carregando mais..."
              HorizontalOptions="Center"
              Margin="10"/>
    </CollectionView.Footer>
</CollectionView>
```


Usando DataTemplate

O **DataTemplate** é o componente XAML responsável por definir a estrutura visual de cada item exibido dentro de uma **CollectionView**.

Sem ele, a **CollectionView** saberia quais dados exibir, por meio da propriedade `ItemsSource`, mas não teria nenhuma instrução sobre como renderizá-los visualmente na tela.

Compreender o **DataTemplate** em profundidade é fundamental para qualquer desenvolvedor .NET MAUI, pois é por meio dele que feeds, catálogos, listas de pedidos e qualquer outra interface baseada em coleções ganham identidade visual e funcionalidade.

A responsabilidade do DataTemplate no MVVM

No contexto do padrão MVVM, o **DataTemplate** pertence exclusivamente à camada de View.

Ele é responsável por três aspectos distintos e bem delimitados: a estrutura visual do item, a apresentação dos dados provenientes da Model, e o encapsulamento das responsabilidades visuais de cada card.

Essa separação garante que a lógica de negócio permaneça na ViewModel, os dados permaneçam na Model, e as decisões visuais como margens, cores, tamanhos de fonte e hierarquia de controles permaneçam exclusivamente no **DataTemplate**.

O resultado prático é um código organizado, reutilizável e de fácil manutenção, onde alterar a aparência de um card no feed não exige nenhuma modificação nas classes C# da aplicação.

Como o BindingContext funciona dentro do DataTemplate

Um detalhe técnico crítico para entender o `DataTemplate` é o mecanismo de `BindingContext` automático que o .NET MAUI aplica a cada item renderizado.

Quando a `CollectionView` percorre a coleção vinculada em `ItemsSource` e cria uma instância visual para cada item, ela automaticamente define o `BindingContext` dessa instância visual para o objeto correspondente da coleção.

Isso significa que, dentro de um `DataTemplate` vinculado a uma `ObservableCollection<Post>`, qualquer expressão `{Binding NomeDaPropriedade}` resolve automaticamente para a propriedade correspondente do objeto `Post` daquele item específico, sem necessidade de qualquer configuração adicional.

Cada instância do `DataTemplate` possui seu próprio `BindingContext` apontando para um `Post` diferente, o que permite que o mesmo template seja reutilizado para renderizar toda a coleção com dados distintos para cada card.

Hierarquia visual do card de feed

Antes de escrever o XAML do `DataTemplate`, é útil visualizar a hierarquia de controles que compõem o card do feed.

O card é estruturado em camadas verticais: um container externo que define as bordas e o fundo, uma imagem de destaque que ocupa a parte superior, um título em negrito abaixo da imagem, um resumo do conteúdo com limite de linhas, e um rodapé horizontal com o nome do autor.



Essa hierarquia reflete diretamente a estrutura XAML que será construída, onde cada nível de aninhamento corresponde a um nível de layout no código.

O container Border

O elemento raiz do **DataTemplate** é o controle **Border**, que substitui o **Frame** legado como container de cards no .NET MAUI moderno.

```

<DataTemplate>
  <Border
    BackgroundColor="{AppThemeBinding Light=White,
                                         Dark=#1E1E1E}"

    Margin="12"
    Padding="0"
    Stroke="#E0E0E0"
    StrokeShape="RoundRectangle 20"
    StrokeThickness="1">

    <VerticalStackLayout Spacing="10">

      <!-- Imagem do feed -->
      <Image
        Aspect="Fill"
        HeightRequest="350"
        Source="{Binding UrlImage}" />

      <!-- Título -->
      <Label
        FontAttributes="Bold"
        FontSize="20"
        LineBreakMode="WordWrap"
        Margin="8,0,8,0"
        Text="{Binding Title}"
        TextColor="{AppThemeBinding Light=Black,
                                     Dark=White}" />

      <!-- Conteúdo -->
      <Label
        FontSize="14"
        LineBreakMode="WordWrap"
        Margin="8,0,8,0"
        MaxLines="3"
        Text="{Binding Content}"
        TextColor="{AppThemeBinding Light=#666666,
                                     Dark=#CCCCCC}" />

      <!-- Rodapé -->
      <HorizontalStackLayout Margin="0,8,0,0">
        <Label
          FontSize="12"
          Margin="8,0,8,0"
          Text="Autor:"
          TextColor="{AppThemeBinding Light=#888888,
                                       Dark=#AAAAAA}" />

        <Label
          FontAttributes="Bold"
          FontSize="12"
          HorizontalOptions="StartAndExpand"
          Margin="8,0,8,10"
          Text="{Binding Author}"
          TextColor="{AppThemeBinding Light=#444444,
                                       Dark=#DDDDDD}" />
      </HorizontalStackLayout>
    </VerticalStackLayout>
  </Border>
</DataTemplate>

```

```

        </VerticalStackLayout>

    </Border>
</DataTemplate>

```

A propriedade **BackgroundColor** utiliza **AppThemeBinding** para adaptar a cor de fundo do card ao tema ativo do dispositivo, exibindo branco no modo claro e um cinza escuro no modo escuro.

A propriedade **Margin** define um espaçamento externo de 12 unidades em todos os lados, criando a separação visual entre cards consecutivos no feed.

A propriedade **Padding** é definida como zero para que a imagem de destaque preencha integralmente a parte superior do card, sem espaçamento interno que quebraria o efeito visual de sangria da imagem.

A propriedade **StrokeShape="RoundRectangle 20"** define o arredondamento dos cantos do card com um raio de 20 unidades, criando a aparência de card moderno amplamente utilizada em interfaces contemporâneas.

A propriedade **StrokeThickness="1"** define a espessura da borda como um pixel, enquanto **Stroke="#E0E0E0"** define sua cor como um cinza claro, adicionando uma delimitação sutil ao card.

A imagem de destaque

Dentro do **VerticalStackLayout**, o primeiro elemento é o controle **Image**, responsável por exibir a imagem principal do post.

A propriedade **Source** recebe o binding **{Binding UrlImage}**, que aponta para a URL remota da imagem armazenada na propriedade **UrlImage** do objeto **Post**.

A propriedade **Aspect="Fill"** instrui o controle a preencher completamente o espaço disponível, recortando a imagem se necessário para evitar espaços em branco nas bordas.

A propriedade **HeightRequest="350"** define a altura fixa da imagem em 350 unidades independentes de dispositivo, garantindo que todos os cards do feed tenham a mesma proporção visual independentemente da dimensão original de cada imagem.

O .NET MAUI realiza o carregamento de imagens remotas de forma assíncrona, o que significa que o card é exibido imediatamente enquanto a imagem é baixada em segundo plano, sem bloquear a interface.

O título e o conteúdo

O **Label** de título utiliza **FontAttributes="Bold"** e **FontSize="20"** para criar uma hierarquia tipográfica clara entre o título e o texto de conteúdo.

A propriedade **LineBreakMode="WordWrap"** garante que títulos longos quebrem linha respeitando os limites da palavra, em vez de cortar o texto no meio de uma palavra.

O **Label** de conteúdo define **MaxLines="3"** para limitar a prévia do texto a três linhas, mantendo os cards com altura consistente independentemente do tamanho do conteúdo de cada post.

A propriedade **TextColor** em ambos os labels utiliza **AppThemeBinding** para adaptar as cores ao tema do dispositivo, garantindo legibilidade tanto no modo claro quanto no modo escuro sem necessidade de lógica adicional no código C#.

O rodapé do card

A seção de rodapé utiliza um `HorizontalStackLayout` para posicionar o rótulo estático “Autor:” e o nome dinâmico do autor lado a lado na mesma linha.

O primeiro `Label` exibe o texto fixo “Autor:” com uma cor mais suave, servindo como rótulo visual para a informação que o segue.

O segundo `Label` utiliza `{Binding Author}` para exibir o nome do autor proveniente do objeto `Post`, com `FontAttributes="Bold"` para destacá-lo em relação ao rótulo.

A propriedade `HorizontalOptions="StartAndExpand"` no `Label` do autor faz com que ele ocupe todo o espaço horizontal restante à sua esquerda, empurrando qualquer elemento seguinte para a extremidade direita do rodapé, o que seria útil ao adicionar uma data de publicação ou um ícone de ação.

Integração com a CollectionView

O `DataTemplate` isolado não tem utilidade por si só: ele precisa ser declarado dentro da propriedade `ItemTemplate` da `CollectionView` para que o componente saiba como renderizar cada item da coleção.

```
<CollectionView ItemsSource="{Binding Posts}">
  <CollectionView.ItemTemplate>
    <DataTemplate>
      <Border
        BackgroundColor="{AppThemeBinding Light=White,
                                             Dark=#1E1E1E}"
        Margin="12"
        Padding="0"
        Stroke="#E0E0E0"
        StrokeShape="RoundRectangle 20"
        StrokeThickness="1">

        <VerticalStackLayout Spacing="10">
```

```

<Image
    Aspect="Fill"
    HeightRequest="350"
    Source="{Binding UrlImage}" />
<!-- Título -->
<Label
    FontAttributes="Bold"
    FontSize="20"
    LineBreakMode="WordWrap"
    Margin="8,0,8,0"
    Text="{Binding Title}"
    TextColor="{AppThemeBinding Light=Black,
                                Dark=White}"
/>

<!-- Conteúdo -->
<Label
    FontSize="14"
    LineBreakMode="WordWrap"
    Margin="8,0,8,0"
    MaxLines="3"
    Text="{Binding Content}"
    TextColor="{AppThemeBinding
Light=#666666,
Dark=#CCCCCC}" />

<!-- Rodapé -->
<HorizontalStackLayout Margin="0,8,0,0">
    <Label
        FontSize="12"
        Margin="8,0,8,0"
        Text="Autor:"
        TextColor="{AppThemeBinding
Light=#888888,
Dark=#AAAAAA}" />

        <Label
            FontAttributes="Bold"
            FontSize="12"
            HorizontalOptions="StartAndExpand"
            Margin="8,0,8,10"
            Text="{Binding Author}"
            TextColor="{AppThemeBinding
Light=#444444,
Dark=#DDDDDD}" />
    </HorizontalStackLayout>
</VerticalStackLayout>
</Border>
</DataTemplate>
</CollectionView.ItemTemplate>
</CollectionView>

```


A **CollectionView** percorre cada objeto **Post** da **ObservableCollection<Post>** exposta pela **FeedViewModel**, cria uma instância da estrutura visual definida no **DataTemplate** para cada um, define o **BindingContext** da instância para o **Post** correspondente e renderiza o resultado na tela.

Vinculando a ViewModel à página

Para que todos os bindings declarados no XAML funcionem corretamente, a **FeedViewModel** precisa estar associada ao **BindingContext** da página.

```
namespace Feed;

public partial class MainPage : ContentPage
{
    public MainPage()
    {
        InitializeComponent();
        BindingContext = new FeedViewModel(); //diz que vai usar a
        feedviewmodel para binding context da pagina
    }
}
```

Com essa configuração, o binding **{Binding Posts}** na propriedade **ItemsSource** da **CollectionView** resolve corretamente para a **ObservableCollection<Post>** da **FeedViewModel**, e todos os bindings internos ao **DataTemplate** resolvem para as propriedades de cada **Post** individualmente.

Resultado visual

Ao executar o aplicativo com a estrutura completa configurada, o feed exibe os cards com a imagem de destaque ocupando a parte superior, o título em negrito, o resumo do conteúdo limitado a três linhas e o nome do autor no rodapé.

Cada card responde automaticamente ao tema do dispositivo, alternando entre as cores do modo claro e do modo escuro sem

nenhuma lógica adicional, graças ao uso consistente de AppThemeBinding em todas as propriedades de cor do DataTemplate.



Evolução do DataTemplate

A estrutura apresentada neste capítulo é intencionalmente minimalista para facilitar a compreensão dos conceitos fundamentais.

Em capítulos subsequentes, esse `DataTemplate` será expandido com a adição de tags coloridas em formato de pill, implementadas por meio de `IValueConverter` que transformam uma string de categorias separadas por vírgula em uma coleção de badges visuais renderizados por um `BindableLayout`.

Essa evolução demonstra uma das maiores vantagens do `DataTemplate`: sua estrutura pode crescer incrementalmente, adicionando novos elementos visuais e comportamentos sem necessidade de reescrever o que já foi construído.

Converters

O sistema de binding do .NET MAUI é capaz de conectar propriedades da ViewModel diretamente a propriedades visuais da View com grande eficiência.

No entanto, existe uma categoria de situações onde o dado disponível na ViewModel não está no formato exato que a interface precisa para renderizar corretamente.

Nesses casos, transformar o dado diretamente na ViewModel para atender a uma necessidade visual seria uma violação do princípio de separação de responsabilidades do MVVM, contaminando a camada de lógica com decisões que pertencem exclusivamente à interface.

O mecanismo criado pelo .NET para resolver esse problema de forma elegante e desacoplada é a interface `IValueConverter`.

O que é o `IValueConverter`

A interface `IValueConverter`, disponível no namespace `Microsoft.Maui.Controls`, define um contrato de transformação de valores que ocorre diretamente na camada de binding, entre o momento em que o dado sai da `ViewModel` e o momento em que ele é aplicado a uma propriedade visual. Ele funciona como uma camada de tradução posicionada no meio do fluxo de dados: recebe um valor de entrada proveniente da `ViewModel`, aplica uma transformação definida pelo desenvolvedor, e entrega o valor transformado para a propriedade visual de destino. Essa transformação ocorre de forma transparente para a `ViewModel`, que continua expondo seus dados no formato natural do domínio, e de forma transparente para a `View`, que recebe exatamente o tipo e o valor que precisa para renderizar corretamente.

Quando utilizar um converter

A decisão de utilizar um `IValueConverter` deve ser tomada sempre que existir uma incompatibilidade de formato ou tipo entre o dado disponível na `ViewModel` e o valor esperado pela propriedade visual de destino. Os casos mais comuns incluem a conversão de um valor booleano em uma cor para destacar visualmente um estado ativo ou inativo, a conversão de um valor booleano na propriedade `IsVisible` de um controle para exibir ou ocultar elementos com base em condições da `ViewModel`, a formatação de um `DateTime` em uma string legível como “há 3 horas” ou “dd/MM/yyyy”, a conversão de um número decimal em uma representação monetária formatada com símbolo de moeda e casas decimais, a conversão de um valor de enum em uma string amigável para exibição ao usuário, a inversão de um valor booleano para casos onde a lógica da `ViewModel` e a necessidade visual são opostas, e a transformação de dados estruturados em representações visuais como a conversão de uma string de tags em uma lista de badges coloridos. O critério central é manter a

ViewModel completamente livre de lógica visual, concentrando qualquer transformação que sirva exclusivamente à interface no converter correspondente.

Adicionando a propriedade Tags à Model

Para demonstrar o uso de converters em um cenário concreto, a classe **Post** recebe uma nova propriedade chamada **Tags**, que armazenará as categorias do post como uma string única com valores separados por vírgula.

```
public class Post
{
    public string Author { get; set; }
    public string Title { get; set; }
    public string Content { get; set; }
    public string UrlImage { get; set; }

    //Nova propriedade para tags e exemplo de conveters
    public string Tags { get; set; }
}
```

O converter StringToListConverter

O primeiro converter necessário é responsável por transformar a string de tags separadas por vírgula em uma **List<string>**, que será consumida pelo **BindableLayout** no XAML.

```
public class StringToListConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object
parameter, CultureInfo culture)
    {
        if (value is string tags && !string.
IsNullOrWhiteSpace(tags))
            return tags.Split(',').Select(t => t.Trim()).ToList();

        return new List<string>();
    }

    public object ConvertBack(object value, Type targetType, object
parameter, CultureInfo culture)
        => throw new NotImplementedException();
}
```

O método **Convert** recebe o valor da propriedade **Tags** como parâmetro **value** e verifica se ele é uma string não nula e não vazia. Quando a condição é satisfeita, o método divide a string pelo caractere de vírgula usando **Split(',', '')**, aplica **Trim()** em cada elemento para remover espaços extras que possam existir antes ou após o separador, e retorna o resultado como **List<string>**. Quando a propriedade **Tags** for nula, vazia ou não for uma string, o método retorna uma lista vazia em vez de nulo, o que evita exceções na interface ao tentar iterar sobre um valor nulo. O método **ConvertBack** lança **NotSupportedException** intencionalmente, pois esse converter é exclusivamente unidirecional: o dado flui da ViewModel para a View, e não existe nenhum cenário onde a interface precisaria converter uma lista de volta para uma string de tags.

O converter TagToColorConverter

O segundo converter é responsável por transformar o valor textual de cada tag individual em uma cor correspondente, que será aplicada ao fundo do badge visual.

```
public class TagToColorConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object
parameter, CultureInfo culture)
    {
        if (value is not string tag)
            return Colors.Gray;

        return tag.ToLower() switch
        {
            "maui" => Colors.Purple,
            "ui" => Colors.Blue,
            "feed" => Colors.Green,
            "backend" => Colors.Orange,
            _ => Colors.Gray
        };
    }

    public object ConvertBack(object value, Type targetType, object
parameter, CultureInfo culture)
        => throw new NotImplementedException();
}
```

O método utiliza pattern matching para verificar se o valor recebido é uma string: caso não seja, retorna cinza como cor padrão defensiva.

Quando o valor é uma string válida, uma expressão switch sobre o valor convertido para minúsculas mapeia cada tag conhecida para sua cor correspondente: roxo para **maui**, azul para **ui**, verde para **feed** e laranja para **backend**.

O caso padrão com `_ => Colors.Gray` garante que tags desconhecidas ou novas categorias ainda recebam uma cor válida, evitando erros de renderização.

O uso de `tag.ToLower()` antes do switch torna a comparação insensível a maiúsculas e minúsculas, o que significa que “MAUI”, “Maui” e “maui” resultarão na mesma cor.

Declarando os converters na ContentPage

Antes de utilizar os converters no XAML, eles precisam ser declarados como recursos na página que os consumirá, por meio do **ContentPage.Resources**.

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage
    x:Class="Feed.MainPage"
    xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:converters="clr-namespace:Feed.Converters"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml">
    <!-- Criamos o ContentPage.Resources para declarar o uso dos
converters -->
    <ContentPage.Resources>
        <ResourceDictionary>
            <converters:StringToListConverter
x:Key="StringToListConverter" />
            <converters:TagToColorConverter
x:Key="TagToColorConverter" />
        </ResourceDictionary>
    </ContentPage.Resources>

    ...
```

A declaração do namespace `xmlns:converters="clr-namespace:Feed.Converters"` informa ao compilador XAML onde encontrar as classes dos converters, que nesse caso estão no namespace `Feed.Converters` do projeto.

Cada converter é registrado no `ResourceDictionary` com uma chave única definida pela propriedade `x:Key`, que será utilizada para referenciá-los nos bindings por meio da extensão `{StaticResource NomeDaChave}`.

Converters podem ser declarados no `ResourceDictionary` da página, de um controle específico, ou no `App.xaml` para uso global em toda a aplicação.

Integrando os converters ao DataTemplate

Com os converters declarados como recursos, o `DataTemplate` pode ser atualizado para exibir as tags em formato de pills coloridos usando `BindableLayout`.

```
<CollectionView.ItemTemplate>
  <DataTemplate>
    <Border
      BackgroundColor="{AppThemeBinding Light=White,
                                           Dark=#1E1E1E}"
      Margin="12"
      Padding="0"
      Stroke="#E0E0E0"
      StrokeShape="RoundRectangle 20"
      StrokeThickness="1">

      <VerticalStackLayout Spacing="10">

        <Image Source="{Binding UriImage}" />

        <!-- Título -->
        <Label
          FontAttributes="Bold"
          FontSize="20"
          LineBreakMode="WordWrap"
          Margin="8,0,8,0"
          Text="{Binding Title}"
          TextColor="{AppThemeBinding Light=Black,
```



```

Dark=White}"
/>

<!-- Conteúdo -->
<Label
    FontSize="14"
    LineBreakMode="WordWrap"
    Margin="8,0,8,0"
    MaxLines="3"
    Text="{Binding Content}"
    TextColor="{AppThemeBinding
Light=#666666,
Dark=#CCCCC}" />

<!-- TAGS -->
<HorizontalStackLayout
    BindableLayout.ItemsSource="{Binding
Tags, Converter={StaticResource StringToListConverter}}"
    Margin="12,4,12,0"
    Spacing="6">

    <BindableLayout.ItemTemplate>
        <DataTemplate>
            <Border
                BackgroundColor="{Binding .,
Converter={StaticResource TagToColorConverter}}"
                HeightRequest="22"
                MinimumWidthRequest="54"
                Padding="10,0"
                StrokeShape="RoundRectangle
11"
                StrokeThickness="0">

                <Label
                    FontAttributes="Bold"
                    FontSize="9"

HorizontalTextAlignment="Center"

                    Text="{Binding .}"
                    TextColor="White"

TextTransform="Uppercase"

VerticalTextAlignment="Center" />
            </Border>
        </DataTemplate>
    </BindableLayout.ItemTemplate>

</HorizontalStackLayout>

<!-- Rodapé -->
<HorizontalStackLayout Margin="0,8,0,0">
    <Label
        FontSize="12"
        Margin="8,0,0,0"

```

```

        Text="Autor:"
        TextColor="{AppThemeBinding
Light=#888888,
Dark=#AAAAAA}" />

        <Label
            FontAttributes="Bold"
            FontSize="12"
            HorizontalOptions="StartAndExpand"
            Margin="4,0,8,10"
            Text="{Binding Author}"
            TextColor="{AppThemeBinding
Light=#444444,
Dark=#DDDDDD}" />
    </HorizontalStackLayout>
</VerticalStackLayout>
</Border>
</DataTemplate>
</CollectionView.ItemTemplate>

```

A propriedade anexada **BindableLayout.ItemsSource** adiciona capacidade de binding de coleção ao **HorizontalStackLayout**, que normalmente não possui esse recurso.

O binding **{Binding Tags, Converter={StaticResource StringToListConverter}}** instrui o sistema a ler a propriedade **Tags** do **Post** atual, passá-la pelo **StringToListConverter** e usar a **List<string>** resultante como fonte de dados para o layout.

Dentro do **BindableLayout.ItemTemplate**, um segundo **DataTemplate** define a aparência de cada pill individual.

O **Border** de cada pill utiliza **{Binding ., Converter={StaticResource TagToColorConverter}}** na propriedade **BackgroundColor**: o **.** representa o próprio objeto do item atual, que nesse contexto é a string da tag, e o **TagToColorConverter** transforma essa string na cor correspondente.

O **Label** dentro do pill exibe o valor da tag com **{Binding .}**, aplica **TextTransform="Uppercase"** para exibir o texto em maiúsculas e centraliza o conteúdo horizontal e verticalmente dentro do badge.

Resultado visual

Com os dois converters integrados ao **DataTemplate**, cada card do feed exibe as tags do post como badges horizontais coloridos posicionados entre o conteúdo e o rodapé.



Cada pill possui cor de fundo determinada automaticamente pelo **TagToColorConverter** com base no valor da tag, texto em branco em maiúsculas e bordas completamente arredondadas com **StrokeShape="RoundRectangle 11"**.

Converters globais versus locais

Os converters declarados no **ResourceDictionary** de uma página estão disponíveis apenas para essa página e seus elementos filhos.

Em projetos onde os mesmos converters são utilizados em múltiplas páginas, o local de declaração recomendado é o **ResourceDictionary** do **App.xaml**, tornando-os globalmente acessíveis a todas as páginas da aplicação sem necessidade de redeclaração.

Outra abordagem, especialmente útil para converters stateless como os apresentados neste capítulo, é declarar a instância diretamente no binding usando a sintaxe de extensão de markup, eliminando completamente a necessidade de declaração prévia no **ResourceDictionary**.

Preparando um aplicativo .NET MAUI para o deploy

Construir um aplicativo funcional é apenas uma parte do trabalho de um desenvolvedor mobile.

A etapa de publicação nas lojas exige uma série de configurações, validações e processos formais que, quando ignorados ou executados de forma incorreta, resultam em rejeição automática do aplicativo ou em problemas críticos detectados apenas em produção.

No .NET MAUI, a preparação para o deploy envolve desde a configuração correta do projeto para build de produção até a conformidade com as políticas específicas de cada loja, passando por assinatura digital, testes em dispositivos reais e monitoramento de erros.

Este artigo cobre cada etapa desse processo de forma sistemática, fornecendo o contexto técnico necessário para publicar um aplicativo com segurança e profissionalismo.

Configuração do projeto para produção

A primeira decisão a tomar antes de qualquer envio às lojas é garantir que o build seja gerado em modo Release e não em modo Debug.

O modo Debug inclui símbolos de depuração, desativa otimizações do compilador e habilita ferramentas de diagnóstico que

aumentam significativamente o tamanho do pacote e reduzem a performance em tempo de execução.

Um aplicativo compilado em Release passa por otimizações de código, tree-shaking de assemblies não utilizados e compressão de recursos, resultando em um pacote menor e mais performático para o usuário final.

Bundle Identifier e Package Name

Cada aplicativo publicado nas lojas precisa de um identificador único que o distingue de todos os outros aplicativos existentes no ecossistema.

No Android, esse identificador é chamado de Package Name e é declarado no **AndroidManifest.xml**. No iOS, ele é chamado de Bundle Identifier e é configurado no **Info.plist**.

```
br.com.suaempresa.nomedoproduto
```

A convenção utilizada é o reverse domain name, onde o domínio registrado pela organização é invertido e concatenado com o nome do produto.

No Brasil, um domínio como **minhaempresa.com.br** resulta em um identificador no formato **br.com.minhaempresa.nomedoproduto**, garantindo unicidade global sem necessidade de um registro centralizado.

Esse identificador é imutável após a publicação: alterá-lo equivale a criar um aplicativo completamente novo nas lojas, perdendo histórico de avaliações, downloads e usuários instalados.

Versão e número de build

O controle de versão de um aplicativo .NET MAUI é gerenciado pelo arquivo de projeto **.csproj**, que centraliza as configurações de versão para todas as plataformas.

```
<ApplicationDisplayVersion>1.0.0</ApplicationDisplayVersion>  
<ApplicationVersion>1</ApplicationVersion>
```

A propriedade **ApplicationDisplayVersion** define a versão visível ao usuário, seguindo o padrão de versionamento semântico com três números separados por ponto representando major, minor e patch.

A propriedade **ApplicationVersion** define o número de build interno, que precisa ser incrementado a cada novo envio às lojas, independentemente de a versão visível ter mudado ou não.

No Android, essas propriedades são mapeadas automaticamente para **versionName** e **versionCode** no **AndroidManifest.xml**. No iOS, são mapeadas para **CFBundleShortVersionString** e **CFBundleVersion** no **Info.plist**.

A grande vantagem do .NET MAUI é justamente essa centralização: alterar os valores no **.csproj** propaga automaticamente para os arquivos de configuração de cada plataforma, eliminando a necessidade de manter valores sincronizados manualmente em múltiplos arquivos.

Ícones e Splash Screen

Todo aplicativo publicado nas lojas precisa fornecer um ícone em múltiplas resoluções exigidas por cada plataforma, uma splash screen para o carregamento inicial e, no Android, um ícone adaptativo que se adapta ao formato e estilo definido pelo fabricante do dispositivo.

O .NET MAUI simplifica esse processo por meio de pastas convencionadas no projeto:

```
Resources/AppIcon  
Resources/Splash
```

Ao posicionar os arquivos de ícone e splash screen nessas pastas, o processo de build do .NET MAUI gera automaticamente todas as variações de tamanho exigidas por cada plataforma.

O formato recomendado para ícones e splash screens no .NET MAUI é o SVG, e não PNG.

Arquivos SVG são vetoriais, o que significa que o processo de build os redimensiona para qualquer resolução sem perda de qualidade, eliminando a necessidade de manter dezenas de arquivos PNG em tamanhos diferentes que precisariam ser atualizados manualmente a cada alteração visual.

Permissões e privacidade

Ambas as plataformas exigem que o aplicativo declare explicitamente quais recursos do dispositivo ele precisa acessar, e essa declaração deve ser justificada tanto tecnicamente quanto nas políticas de privacidade.

No Android, as permissões são declaradas no `AndroidManifest.xml` e incluem recursos como acesso à internet, câmera, localização, Bluetooth e armazenamento local.

```
AndroidManifest.xml
```

Cada permissão declarada deve ter uma justificativa técnica clara, pois a equipe de revisão da Google Play pode solicitar explicações sobre o motivo pelo qual o aplicativo necessita de cada recurso.

No iOS, o processo é ainda mais rigoroso.

Além de declarar as permissões, o **Info.plist** exige que cada permissão contenha uma descrição textual explicando ao usuário por que o aplicativo precisa daquele acesso específico:

```
NSCameraUsageDescription  
NSLocationWhenInUseUsageDescription  
NSMicrophoneUsageDescription
```

Essas descrições são exibidas diretamente ao usuário no momento em que o sistema operacional solicita autorização para o recurso.

A ausência de qualquer uma dessas descrições para recursos utilizados pelo aplicativo resulta em rejeição imediata pela Apple, sem possibilidade de recurso até que a descrição seja adicionada e um novo build seja submetido.

Assinatura do aplicativo

Nenhum aplicativo pode ser instalado em um dispositivo ou publicado em uma loja sem estar assinado digitalmente com um certificado válido.

A assinatura digital garante a autenticidade do pacote, confirmando que ele foi gerado pela organização ou desenvolvedor responsável e que não foi adulterado após a geração.

No Android, a assinatura é realizada com um Keystore, que é um arquivo de chave criptográfica gerado pelo desenvolvedor:

```
keytool -genkeypair -v -keystore myapp.keystore
```

Esse arquivo precisa ser armazenado com segurança e nunca

deve ser perdido ou comprometido, pois é o único mecanismo que permite publicar atualizações do aplicativo na Google Play.

A Google exige que o pacote final seja enviado no formato Android App Bundle:

Android App Bundle (.aab)

O formato **.aab** permite que a Google Play entregue ao usuário apenas os recursos necessários para o dispositivo específico, reduzindo o tamanho do download em comparação ao formato **.apk** tradicional.

No ecossistema Apple, o processo de assinatura é mais elaborado e envolve cinco etapas interdependentes: a criação de uma conta Apple Developer com o pagamento da taxa anual, a geração de certificados de distribuição no portal da Apple, a criação de Provisioning Profiles que associam o Bundle Identifier aos certificados e dispositivos autorizados, o registro do App Identifier no portal, e a assinatura do build final utilizando esses artefatos.

As ferramentas envolvidas nesse processo são o Xcode para gerenciamento de certificados e profiles, o Transporter para upload do pacote à App Store Connect, e o Visual Studio ou a CLI do .NET para a geração do build assinado.

O arquivo gerado pelo processo de build iOS é o **.ipa**

Testes antes da publicação

Publicar um aplicativo sem uma bateria de testes estruturada é uma das causas mais comuns de crashes em produção e de avaliações negativas nas lojas logo após o lançamento.

Os testes funcionais devem cobrir obrigatoriamente os fluxos de login e autenticação, a navegação entre todas as telas principais,

as integrações com APIs externas, os mecanismos de sincronização de dados, o comportamento do aplicativo sem conexão de rede e o tratamento de erros inesperados sem crash.

Os testes de performance devem monitorar o consumo de memória ao longo de sessões prolongadas, o impacto na bateria durante uso contínuo, o tempo de abertura do aplicativo a partir do estado fechado e a ausência de travamentos durante operações intensas como scroll rápido em listas longas.

Os testes em dispositivos reais são insubstituíveis e precisam cobrir diferentes resoluções de tela, múltiplas versões do Android para garantir compatibilidade com versões mais antigas do sistema, e diferentes modelos de iPhone para validar o comportamento em telas com notch, ilha dinâmica e diferentes tamanhos de display.

Emuladores e simuladores são ferramentas úteis para desenvolvimento ágil, mas não reproduzem com fidelidade o comportamento de sensores, câmeras, conectividade e performance de hardware real.

Tratamento de erros e monitoramento

Um aplicativo publicado sem instrumentação de monitoramento cria uma situação onde problemas em produção são descobertos apenas por meio de avaliações negativas dos usuários nas lojas, que geralmente descrevem os sintomas sem informações técnicas úteis para o diagnóstico.

As ferramentas recomendadas para monitoramento de aplicativos .NET MAUI incluem o Application Insights da Microsoft, que integra nativamente com o ecossistema Azure e oferece rastreamento de eventos, métricas de performance e logs estruturados; o Firebase Crashlytics do Google, que fornece relatórios detalhados de crashes com stack trace completo e agrupamento auto-

mático por tipo de erro; e o Sentry, que combina rastreamento de erros com monitoramento de performance e suporte a múltiplas plataformas.

Qualquer uma dessas ferramentas deve ser integrada e validada antes da publicação, nunca após o primeiro crash em produção.

Política de privacidade

Tanto a Apple App Store quanto a Google Play exigem que todo aplicativo que colete, processe ou transmita dados de usuários possua uma Política de Privacidade pública e acessível.

Esse documento precisa informar de forma clara e compreensível quais dados são coletados durante o uso do aplicativo, como esses dados são armazenados e protegidos, se são compartilhados com terceiros e em que condições, e qual o processo para que o usuário solicite a exclusão de seus dados.

A Política de Privacidade deve estar disponível em uma URL pública que seja fornecida tanto no cadastro do aplicativo nas lojas quanto dentro do próprio aplicativo, geralmente no menu de configurações ou na tela de termos de uso.

A ausência desse documento é motivo de rejeição nas duas lojas, e uma política incompleta ou genérica pode ser sinalizada durante revisões futuras.

Conteúdo da página do aplicativo nas lojas

Além do pacote do aplicativo em si, as lojas exigem uma série de materiais de marketing que compõem a página pública do aplicativo e influenciam diretamente a taxa de conversão de visitantes em instalações.

As informações obrigatórias incluem o nome do aplicativo, uma descrição curta que aparece nos resultados de busca, uma descrição completa com todos os recursos e benefícios, a categoria correta para facilitar a descoberta e a classificação etária baseada no conteúdo.

As imagens obrigatórias são os screenshots do aplicativo em execução, que para o Android devem incluir capturas de um dispositivo telefone e opcionalmente de um tablet, e para o iOS devem incluir capturas de iPhone e iPad quando o aplicativo suportar ambos os formatos.

Materiais opcionais como vídeos de apresentação e banners promocionais aumentam significativamente o impacto visual da página e são recomendados especialmente para lançamentos que investem em campanhas de aquisição de usuários.

Conformidade com as regras das lojas

Antes de submeter o aplicativo para revisão, é fundamental verificar se ele está em conformidade com as políticas específicas de cada loja, pois a rejeição por violação de regras pode levar dias para ser resolvida e atrasar o lançamento.

Os motivos mais comuns de rejeição incluem aplicativos sem funcionalidade real além de exibir conteúdo de um site, coleta de dados sem explicação clara na política de privacidade, solicitação de permissões que não são utilizadas pelo aplicativo, exigência de login obrigatório antes de o usuário ver qualquer conteúdo do aplicativo e crashes reproduzíveis identificados durante a revisão.

A Apple costuma ser significativamente mais rigorosa que a Google em suas revisões, com critérios mais subjetivos relacionados à qualidade da experiência do usuário e ao valor entregue pelo aplicativo.

Build final do aplicativo

Com todas as verificações concluídas, o passo final antes do envio é a geração dos pacotes de produção para cada plataforma.

Para Android, o comando de publicação gera o arquivo `.aab` assinado e otimizado para distribuição:

```
dotnet publish -f net8.0-android -c Release
```

Para iOS, o comando correspondente gera o arquivo `.ipa` assinado com os certificados configurados:

```
dotnet publish -f net8.0-ios -c Release
```

Envio para as lojas

O upload do pacote Android é realizado pelo Google Play Console, onde o processo envolve criar o registro do aplicativo na plataforma, fazer o upload do arquivo `.aab`, preencher todas as informações da página do aplicativo e enviar para revisão.

O tempo médio de aprovação na Google Play varia entre algumas horas e dois dias para novos aplicativos, sendo geralmente mais rápido para atualizações.

O upload do pacote iOS é realizado pelo aplicativo Transporter, que acompanha o Xcode, e o processo envolve criar o registro do aplicativo no App Store Connect, fazer o upload do arquivo `.ipa` pelo Transporter, preencher todos os metadados e submeter para revisão.

O tempo médio de aprovação na Apple App Store é de um a cinco dias úteis para novos aplicativos, com variações dependendo do volume de submissões e da complexidade do aplicativo.

Checklist final de publicação

Antes de enviar o aplicativo às lojas, cada item da lista abaixo deve estar confirmado:

- Build compilado em modo Release
- Bundle Identifier configurado corretamente
- Versão do aplicativo definida e incrementada
- Ícones e splash screen configurados em formato SVG
- Permissões declaradas e justificadas
- Política de privacidade publicada em URL pública
- Certificados de assinatura configurados e validados
- Testes realizados em dispositivos reais
- Ferramenta de monitoramento de erros integrada e ativa
- Screenshots e descrição da página preparados
- Pacote **.aab** ou **.ipa** gerado e validado corretamente

Conclusão

Ao longo deste material, você não apenas leu sobre o .NET MAUI.

Você construiu algo.

Partiu do zero, entendeu o padrão que organiza aplicativos móveis de verdade, implementou uma interface capaz de exibir conteúdo dinâmico com performance, adicionou comportamentos visuais reativos sem contaminar a lógica de negócio, e preparou o aplicativo para o mundo real.

Isso não é pouco. É exatamente o ponto de partida que separa quem faz experimentos de quem entrega produtos.

O que foi construído

O feed que você implementou neste livro tem a mesma estrutura que sustenta aplicativos com milhões de usuários.

A **FeedViewModel** expõe dados via **ObservableCollection<Post>**, notifica a interface automaticamente quando o estado muda, encapsula ações em **ICommand** e herda um contrato de reatividade da **BaseViewModel** que escala para projetos de qualquer tamanho sem exigir reescrita.

A **CollectionView** renderiza os itens com virtualização nativa, garantindo que o feed continue fluido independentemente do volume de dados, e organiza header, body e footer em um único contexto de scroll sem a necessidade de nenhum **ScrollView** externo.

O **DataTemplate** define a aparência de cada card de forma to-

talmente declarativa em XAML, adaptando cores ao tema do dispositivo via **AppThemeBinding** e separando estrutura visual de dados com a precisão que o padrão MVVM exige.

Os **IValueConverter** transformam dados brutos em representações visuais ricas, como as tags em formato de pill com cores semânticas, sem adicionar uma única linha de lógica de apresentação à ViewModel.

E o processo de deploy foi mapeado do início ao fim: desde a configuração de versão centralizada no **.csproj**, passando pela assinatura digital com Keystore e Provisioning Profiles, até o envio para o Google Play Console e a App Store Connect com todos os materiais e políticas necessários.

O que o .NET MAUI representa para desenvolvedores .NET

Por muito tempo, desenvolvedores C# que precisavam criar aplicativos móveis enfrentavam uma escolha difícil: aprender Swift e Kotlin, adotar React Native, ou aceitar as limitações de plataformas híbridas que sacrificavam performance e acesso nativo.

O .NET MAUI elimina essa tensão. Com ele, o conhecimento que você já tem em C#, LINQ, injeção de dependência, testes unitários e arquitetura limpa se aplica diretamente ao desenvolvimento mobile e desktop. A curva de aprendizado existe, principalmente em relação ao XAML e aos detalhes específicos de cada plataforma, mas ela é significativamente menor do que reaprender uma stack completamente diferente.

Empresas que já possuem equipes .NET estão adotando o MAUI justamente por esse motivo: o mesmo desenvolvedor que man-

tém a API pode contribuir com o aplicativo móvel, usando as mesmas práticas, as mesmas ferramentas e o mesmo ecossistema.

O que este livro não cobriu

Este material foi construído com uma premissa deliberada: profundidade nos fundamentos, não abrangência superficial de todos os recursos. Há um universo de tópicos que vão além do que foi apresentado aqui e que fazem parte da jornada de qualquer desenvolvedor .NET MAUI que queira evoluir.

Navegação entre páginas com Shell ou NavigationPage, gerenciamento de estado em aplicativos com múltiplas telas, injeção de dependência com **Microsoft.Extensions.DependencyInjection** no **Mauiprogram.cs**, animações e transições visuais nativas, acesso a recursos do dispositivo como câmera, GPS e Bluetooth via **IMediaPicker** e **IGeolocation**, integração com APIs REST utilizando **HttpClient** e serialização com **System.Text.Json**, cache de imagens com bibliotecas especializadas, testes unitários e de integração para ViewModels e serviços, e a construção de componentes visuais customizados com **GraphicsView** e **SKCanvas**.

Cada um desses tópicos tem profundidade suficiente para um material próprio. O que este livro forneceu foi o alicerce sobre o qual todos eles se apoiam.

Próximos passos concretos

O melhor que você pode fazer agora é não parar.

Pegue o aplicativo de feed que foi construído ao longo deste material e evolua ele. Substitua os dados estáticos da **FeedViewModel** por uma chamada real a uma API REST utilizando **HttpClient** com **async/await** e um indicador de carregamento via propriedade `IsLoading` notificável.

Adicione paginação ao feed utilizando **RemainingItemsThreshold** e **RemainingItemsThresholdReachedCommand** para carregar novos posts automaticamente quando o usuário chegar próximo ao final da lista.

Implemente pull to refresh encapsulando a **CollectionView** em um **RefreshView** vinculado a um **ICommand** de recarregamento na ViewModel. Adicione uma segunda tela de detalhe do post e explore navegação com Shell, passando dados entre páginas via query parameters ou via serviço de estado compartilhado.

Refatore a injeção da **FeedViewModel** para usar o container de DI nativo do MAUI no **MauiProgram.cs**, removendo a instancição direta do construtor da **MainPage**.

Escreva testes unitários para a **FeedViewModel** verificando a contagem de posts, os estados de carregamento e o comportamento do comando de recarregamento. Cada uma dessas evoluções vai solidificar o entendimento dos conceitos apresentados aqui e adicionar camadas de profissionalismo ao que foi construído.

Uma palavra sobre o ecossistema

O .NET MAUI não existe sozinho.

Ele é parte de um ecossistema maior que inclui o **Community-Toolkit.Mvvm** para eliminar boilerplate de ViewModels com source generators, o **CommunityToolkit.Maui** para controles e comportamentos adicionais não presentes no framework base, o **Maui.Controls.Maps** para integração de mapas, a **SQLite-net** para persistência local, e uma comunidade crescente de desenvolvedores que contribuem com bibliotecas, tutoriais e soluções para os problemas mais comuns.

A Microsoft tem investido consistentemente no MAUI desde seu lançamento, com atualizações regulares que endereçam performance, estabilidade e novos recursos de plataforma.

Apostar no .NET MAUI como plataforma principal para desenvolvimento mobile é uma decisão tecnicamente sólida e respaldada por um ecossistema maduro.

Código-fonte do projeto

O aplicativo completo construído ao longo deste livro está disponível no GitHub:

<https://github.com/Saccomani/FeedBalta>

Use o repositório como referência, como ponto de partida para suas próprias experimentações, e como material de comparação quando algo não estiver funcionando como esperado.

Consideração final

Desenvolver para múltiplas plataformas com uma única base de código em C# deixou de ser promessa para se tornar realidade madura e adotada em produção.

O .NET MAUI exige conhecimento sólido de C# e disposição para entender as nuances de XAML e do padrão MVVM.

Mas quem investe nesse aprendizado ganha a capacidade de construir, manter e evoluir aplicativos para Android, iOS, Windows e macOS com as mesmas ferramentas, as mesmas práticas e a mesma stack que já domina.

O que foi aprendido aqui é o começo.

O próximo passo é seu.

